

Compiladores 2026 — Fase 2

Frontend/Backend, Código Intermedio, Optimización y Visualización

1. Resumen

En esta fase se reorganizó el compilador en **frontend** (análisis) y **backend** (transformación), se incorporó la generación de **código intermedio** en formato *Three-Address Code* (TAC) y se construyó un **optimizador** que trabaja sobre ese TAC. Además se extendió la interfaz web para mostrar el TAC original, el optimizado y la traza del optimizador.

A continuación se presenta el código real del proyecto, archivo por archivo, explicando qué hace cada bloque.

2. Separación Frontend / Backend

2.1. Estructura física

```
files (1)/
|-- bridge.py                # orquestador (CLI)
|-- intermedio.py           # contrato compartido (Quad + formatter)
|-- frontend/
|   |-- __init__.py
|   |-- lexer.py
|   |-- tabla_simbolos.py
|   |-- parser.py
|   '-- generador_intermedio.py
'-- backend/
    |-- __init__.py
    '-- optimizador.py
```

2.2. frontend/__init__.py — qué exporta el frontend

```
from .lexer import Lexer
from .tabla_simbolos import TablaSimbolos, Simbolo
from .parser import build_tree, get_kids, node_label, TIPOS_DATO,
    RESERVED
from .generador_intermedio import GeneradorTAC

__all__ = [
    "Lexer",
    "TablaSimbolos", "Simbolo",
    "build_tree", "get_kids", "node_label", "TIPOS_DATO", "RESERVED",
    "GeneradorTAC",
]
```

Qué hace: agrupa todos los componentes del análisis en un solo paquete y los re-exporta. El `bridge.py` hace `from frontend import Lexer, build_tree, GeneradorTAC` y no se entera de cómo está organizado por dentro.

2.3. backend/__init__.py — qué exporta el backend

```
from .optimizador import optimizar

__all__ = ["optimizar"]
```

Qué hace: expone únicamente la función `optimizar(quads)`. El backend hoy sólo optimiza, pero la interfaz pública queda lista para agregar generadores de código objeto en fases siguientes sin tocar el resto.

3. Frontera compartida: `intermedio.py`

Este archivo es el **único** módulo que ambos lados conocen. Define la estructura de un cuádruplo y un *pretty-printer* para mostrarlo.

3.1. La dataclass `Quad`

```
@dataclass
class Quad:
    op: str
    arg1: Optional[str] = None
    arg2: Optional[str] = None
    dest: Optional[str] = None

    def as_tuple(self):
        return (self.op,
                self.arg1 if self.arg1 is not None else "_",
                self.arg2 if self.arg2 is not None else "_",
                self.dest if self.dest is not None else "_")
```

Qué hace:

- `op`: el operador (+, =, goto, label, ...).
- `arg1`, `arg2`: los dos operandos fuente (cualquiera puede faltar).
- `dest`: el destino. Para asignaciones y operaciones es el nombre/temporal donde queda el resultado; para saltos es la etiqueta a la que se salta.
- `as_tuple()`: devuelve la 4-tupla con "_" cuando hay None, útil para imprimir o serializar.

3.2. `formatear_tac` — pretty-printer para la UI

```
def formatear_tac(quads):
    filas = []
    n = 0
    for q in quads:
        op, a1, a2, d = q.op, q.arg1, q.arg2, q.dest
        if op == "label":
            instr = f"{d}:"
            etiqueta = d
        elif op == "goto":
            instr = f"goto {d}"
        elif op == "if_false":
            instr = f"ifFalse {a1} goto {d}"
        elif op == "=":
            instr = f"{d} = {a1}"
        elif op == "print":
            instr = f"print {a1}"
        elif op == "param":
            instr = f"param {a1}"
        elif op == "call":
            instr = f"{d} = call {a1}, {a2}"
        elif op == "!":
            instr = f"{d} = !{a1}"
        else:
            instr = f"{d} = {a1} {op} {a2}" # binarias
        n += 1
    filas.append({"n": n, "etiqueta": etiqueta, "instruccion": instr,
```

```

        "op": op, "arg1": a1 or "-", "arg2": a2 or "-",
        "dest": d or "-"}
    return filas

```

Qué hace: recorre la lista de cuádruplos y produce una lista de diccionarios listos para que el JS de la UI los pinte como tabla o como texto plano. Cada *op* tiene su propio formato legible (ifFalse \$t2 goto \$L1, c = \$t1, ...) sin perder los campos crudos (op, arg1, arg2, dest).

4. Generación del TAC: frontend/generador_intermedio.py

4.1. Mapa de operadores

```

_BIN_TIPOS = {
    "MAS": "+", "MENOS": "-", "MULT": "*", "DIV": "/", "MOD": "%",
    "IGUAL": "==", "DIFERENTE": "!=",
    "MENOR": "<", "MAYOR": ">",
    "MENOR_IGUAL": "<=", "MAYOR_IGUAL": ">=",
    "Y_LOGICO": "&&", "O_LOGICO": "||",
}

```

Qué hace: traduce el tipo del token (como lo emite el lexer) al símbolo que va a quedar dentro del cuádruplo. Por ejemplo, el token MAS se vuelve "+" en el TAC.

4.2. Estado del generador y helpers

```

class GeneradorTAC:
    def __init__(self):
        self.codigo: list[Quad] = []
        self._t = 0
        self._l = 0

    def _nuevo_temp(self) -> str:
        self._t += 1
        return f"$t{self._t}"

    def _nueva_etiq(self) -> str:
        self._l += 1
        return f"$L{self._l}"

    def emit(self, op, arg1=None, arg2=None, dest=None):
        self.codigo.append(Quad(op, arg1, arg2, dest))

```

Qué hace:

- *self.codigo*: lista de cuádruplos que se va construyendo.
- *_nuevo_temp()*: devuelve nombres frescos \$t1, \$t2, ... para guardar resultados intermedios.
- *_nueva_etiq()*: devuelve nombres frescos \$L1, \$L2, ... para destinos de salto.
- *emit()*: empuja un cuádruplo al final de la lista.

4.3. Punto de entrada y dispatcher

```

def generar(self, ast) -> list[Quad]:
    self.codigo = []
    self._t = 0
    self._l = 0
    if ast:
        self._gen(ast)
    return self.codigo

```

```
def _gen(self, node):
    if not node:
        return None
    t = node.get("type", "")
    method = getattr(self, f"_g_{t}", self._g_default)
    return method(node)
```

Qué hace: generar resetea el estado y arranca el recorrido. `_gen` es el **dispatcher**: mira el campo `type` del nodo del AST y llama al método `_g_<Tipo>`, p.ej. `_g_If` para un nodo `If`. Es un visitor implementado con `getattr`, sin `if/elif` repetitivos.

4.4. Expresiones binarias — patrón post-orden

```
if t == "BinaryOp":
    izq = self._g_expr(node["left"])      # subarbol izquierdo
    der = self._g_expr(node["right"])     # subarbol derecho
    tmp = self._nuevo_temp()
    op_tipo = node["op"]["tipo"]
    op = _BIN_TIPOS.get(op_tipo, node["op"]["valor"])
    self.emit(op, izq, der, tmp)
    return tmp
```

Qué hace: traduce `a op b` en TAC. Primero genera el código del operando izquierdo y obtiene el lugar (variable o temporal) donde quedó su resultado; lo mismo con el derecho; pide un temporal nuevo y emite el cuádruplo `tmp = izq op der`. El temporal devuelto se encadena en operaciones más grandes: `(a+b)*c` produce `$t1 = a + b` y luego `$t2 = $t1 * c`.

4.5. Estructuras de control

4.5.1. If / else

```
def _g_If(self, node):
    cond = self._g_expr(node.get("cond"))
    L_else = self._nueva_etiq()
    L_end = self._nueva_etiq() if node.get("elseBody") else L_else

    self.emit("if_false", cond, None, L_else)
    self._gen(node.get("body"))
    if node.get("elseBody"):
        self.emit("goto", None, None, L_end)
        self.emit("label", None, None, L_else)
        self._gen(node["elseBody"])
        self.emit("label", None, None, L_end)
    else:
        self.emit("label", None, None, L_else)
```

Qué hace: si la condición es falsa salta a `L_else`; si no, ejecuta el cuerpo. Cuando hay `else`, después del cuerpo se hace `goto L_end` para no caer en el bloque `else`. Las dos etiquetas marcan los puntos de aterrizaje.

4.5.2. While

```
def _g_While(self, node):
    L_inicio = self._nueva_etiq()
    L_fin = self._nueva_etiq()
    self.emit("label", None, None, L_inicio)
    cond = self._g_expr(node.get("cond"))
    self.emit("if_false", cond, None, L_fin)
    self._gen(node.get("body"))
    self.emit("goto", None, None, L_inicio)
```

```
self.emit("label", None, None, L_fin)
```

Qué hace: pone una etiqueta al inicio del lazo, evalúa la condición, salta al final si es falsa, ejecuta el cuerpo y vuelve al inicio con un `goto`. `L_fin` es el destino tanto del `ifFalse` como del eventual `break` (no implementado en esta fase).

4.5.3. Print

```
def _g_Print(self, node):
    arg = self._g_expr(node.get("arg"))
    self.emit("print", arg, None, None)
```

Qué hace: genera el código para evaluar el argumento (puede ser una expresión compleja) y emite un cuádruplo `print arg`.

4.6. Ejemplo de salida

Código fuente:

```
int a = 3;
int b = 4;
int c = a + b;
if (c > 5) { print(c); } else { print(0); }
```

TAC producido por GeneradorTAC:

```
1:  a = 3
2:  b = 4
3:  $t1 = a + b
4:  c = $t1
5:  $t2 = c > 5
6:  ifFalse $t2 goto $L1
7:  print c
8:  goto $L2
9:  $L1:
10: print 0
11: $L2:
```

5. Optimización: backend/optimizador.py

5.1. Tabla de operaciones evaluables en compile-time

```
_BIN_OPS = {
    "+": lambda a, b: a + b,
    "-": lambda a, b: a - b,
    "*": lambda a, b: a * b,
    "/": lambda a, b: a / b if isinstance(a, float) or isinstance(b, float)
        else a // b,
    "%": lambda a, b: a % b,
    "==": lambda a, b: a == b,
    "!=": lambda a, b: a != b,
    "<": lambda a, b: a < b,    ">": lambda a, b: a > b,
    "<=": lambda a, b: a <= b, ">=": lambda a, b: a >= b,
    "&&": lambda a, b: bool(a) and bool(b),
    "||": lambda a, b: bool(a) or bool(b),
}
```

Qué hace: mapea cada operador del TAC a su implementación Python. Lo usa el *constant folding* para evaluar la operación si los dos argumentos resultan ser constantes literales.

5.2. Pasada 1 — Constant Folding y simplificación algebraica

```
def _constant_folding(codigo):
    nuevo = []
    cambio = False
    for q in codigo:
        if q.op in _BIN_OPS:
            a1, a2 = q.arg1, q.arg2
            # Identidades algebraicas
            if q.op == "+" and a1 == "0":
                nuevo.append(Quad("=", a2, None, q.dest)); cambio = True;
                continue
            if q.op == "+" and a2 == "0":
                nuevo.append(Quad("=", a1, None, q.dest)); cambio = True;
                continue
            if q.op == "-" and a2 == "0":
                nuevo.append(Quad("=", a1, None, q.dest)); cambio = True;
                continue
            if q.op == "*" and (a1 == "1"):
                nuevo.append(Quad("=", a2, None, q.dest)); cambio = True;
                continue
            if q.op == "*" and (a2 == "1"):
                nuevo.append(Quad("=", a1, None, q.dest)); cambio = True;
                continue
            if q.op == "*" and (a1 == "0" or a2 == "0"):
                nuevo.append(Quad("=", "0", None, q.dest)); cambio = True;
                continue
            # Folding numerico
            if _es_numero(a1) and _es_numero(a2):
                r = _BIN_OPS[q.op](_to_num(a1), _to_num(a2))
                nuevo.append(Quad("=", _fmt(r), None, q.dest))
                cambio = True; continue
        nuevo.append(q)
    return nuevo, cambio
```

Qué hace: para cada cuádruplo binario, primero detecta *identidades algebraicas* ($x+0$, $x*1$, $x*0\dots$) y reemplaza la operación por una asignación directa. Después, si los dos operandos son constantes numéricas, calcula el resultado y reemplaza el cuádruplo por `dest = <resultado>`. Devuelve el nuevo código y un flag `cambio`.

5.3. Pasada 2 — Constant / Copy Propagation

```
def _propagation(codigo):
    nuevo = []
    env = {} # variable/temp -> valor conocido
    cambio = False

    def reemplazar(x):
        return env.get(x, x) if x is not None else x

    for q in codigo:
        if q.op == "label":
            env = {} # se invalida en bordes de bloques
            nuevo.append(q); continue
        if q.op in ("goto", "if_false", "if_true"):
            nuevo.append(Quad(q.op, reemplazar(q.arg1), reemplazar(q.arg2), q.dest))
            env = {}; continue
        if q.op == "=":
            valor = reemplazar(q.arg1)
            nuevo.append(Quad("=", valor, None, q.dest))
            env[q.dest] = valor # ahora dest "es" valor
            continue
```

```

# Resto: reemplazar argumentos
a1 = reemplazar(q.arg1); a2 = reemplazar(q.arg2)
nuevo.append(Quad(q.op, a1, a2, q.dest))
if q.dest is not None:
    env.pop(q.dest, None)
return nuevo, cambio

```

Qué hace: mantiene un diccionario `env` con los valores que conoce de cada variable/temporal. Cuando ve `x = 5`, recuerda `env["x"] = "5"`. Cuando ve un uso posterior de `x`, lo reemplaza por "5". El entorno se **invalida** al cruzar una etiqueta o un salto, porque ahí pueden venir valores de otros caminos. Cualquier reasignación de `dest` también borra entradas viejas asociadas a ese destino.

5.4. Pasada 3 — Branch Pruning

```

def _branch_pruning(codigo):
    nuevo = []
    cambio = False
    for q in codigo:
        if q.op == "if_false":
            if q.arg1 in ("true", "True", "1"):
                cambio = True; continue          # ifFalse true ->
                elimina
            if q.arg1 in ("false", "False", "0"):
                nuevo.append(Quad("goto", None, None, q.dest))
                cambio = True; continue          # ifFalse false ->
                goto
        if q.op == "if_true":
            if q.arg1 in ("false", "False", "0"):
                cambio = True; continue
            if q.arg1 in ("true", "True", "1"):
                nuevo.append(Quad("goto", None, None, q.dest))
                cambio = True; continue
    nuevo.append(q)
    return nuevo, cambio

```

Qué hace: cuando la pasada de *propagation* dejó la condición de un salto convertida en literal (`true/false`), aquí se elimina el salto inútil o se convierte en un `goto` incondicional.

5.5. Pasada 4 — Dead-Code Elimination

```

def _dead_code(codigo):
    leídos = set()
    for q in codigo:
        for a in (q.arg1, q.arg2):
            if a is not None: leídos.add(a)
        if q.op in ("goto", "if_false", "if_true") and q.dest:
            leídos.add(q.dest)

    nuevo = []
    cambio = False
    for q in codigo:
        if q.op == "=" or q.op in _BIN_OPS or q.op == "!":
            if q.dest and _es_temp(q.dest) and q.dest not in leídos:
                cambio = True; continue          # temporal no leído => fuera
        nuevo.append(q)
    return nuevo, cambio

```

Qué hace: primero escanea *todo* el código y arma el conjunto de nombres que aparecen como `leídos` (en `arg1` o `arg2`, o como destino de un salto). Luego elimina los cuádruplos que escriben

en un **temporal** (\$tn) que nunca se leyó. Sólo se borran temporales: las variables del usuario se conservan aunque parezcan muertas, porque pueden tener efecto observable (print posterior, scope externo, etc.).

5.6. Pasada 5 — Jump Threading

```
def _jump_threading(codigo):
    nuevo = []
    cambio = False
    i = 0
    while i < len(codigo):
        q = codigo[i]
        if (q.op == "goto"
            and i + 1 < len(codigo)
            and codigo[i + 1].op == "label"
            and codigo[i + 1].dest == q.dest):
            cambio = True; i += 1; continue # goto L seguido de label L:
            borra goto
        nuevo.append(q); i += 1

    referenciadas = set() # etiquetas todavia usadas
    for q in nuevo:
        if q.op in ("goto", "if_false", "if_true") and q.dest:
            referenciadas.add(q.dest)
    final = []
    for q in nuevo:
        if q.op == "label" and q.dest not in referenciadas:
            cambio = True; continue # label huérfana
        final.append(q)
    return final, cambio
```

Qué hace: dos limpiezas. Primero, si encuentra un goto L inmediatamente seguido de L:, borra el goto porque el flujo cae igual a la etiqueta. Segundo, calcula qué etiquetas quedaron *referenciadas* y elimina las que ya no apunta nadie.

5.7. Driver: optimizar() — punto fijo

```
def optimizar(codigo, max_iter: int = 12):
    actual = deepcopy(codigo)
    traza = []
    pasadas = [
        ("Constant Folding & Algebraic", _constant_folding),
        ("Constant / Copy Propagation", _propagation),
        ("Branch Pruning", _branch_pruning),
        ("Dead-Code Elimination", _dead_code),
        ("Jump Threading", _jump_threading),
    ]

    for it in range(max_iter):
        cambio_global = False
        for nombre, fn in pasadas:
            antes = len(actual)
            actual, cambio = fn(actual)
            despues = len(actual)
            if cambio:
                traza.append({"iter": it+1, "pasada": nombre,
                             "antes": antes, "despues": despues,
                             "delta": despues - antes})
                cambio_global = True
        if not cambio_global:
            break
    return actual, traza
```


Qué hace: aplica las cinco pasadas en orden, una y otra vez, hasta que una iteración completa no haga ningún cambio (**punto fijo**) o se llegue a `max_iter`. Cada vez que una pasada modifica el código, se anota una entrada en `traza` con el nombre, el tamaño antes/después y el delta. Devuelve (`codigo_optimizado`, `traza`).

5.8. Antes y después

Código fuente:

```
int a = 3;  int b = 4;
int c = a + b;      int d = c * 2;
int e = d - 0;      int f = e * 1;
print(f);
```

TAC original

```
a = 3
b = 4
$t1 = a + b
c = $t1
$t2 = c * 2
d = $t2
$t3 = d - 0
e = $t3
$t4 = e * 1
f = $t4
print f
```

TAC optimizado

```
a = 3
b = 4
c = 7
d = 14
e = 14
f = 14
print 14
```

6. Orquestador: bridge.py

```
from frontend import Lexer, build_tree, GeneradorTAC
from intermedio import formatear_tac
from backend import optimizar

# 1) lexer
lexer = Lexer()
tokens_info, tabla, errores = lexer.analizar(codigo)

# 2) parser -> AST -> TAC
ast = build_tree(tokens_info)
quads = GeneradorTAC().generar(ast)
tac_filas = formatear_tac(quads)

# 3) optimizador
quads_opt, traza = optimizar(quads)
tac_opt_filas = formatear_tac(quads_opt)

# 4) metricas + JSON
metricas = {
    "cuad_orig": len(quads), "cuad_opt": len(quads_opt),
    "reduccion_pct": round(100.0*(len(quads)-len(quads_opt))/len(quads), 1),
    "temps_orig": _contar_temps(quads), "temps_opt": _contar_temps(quads_opt),
}
print(json.dumps({
    "tokens": tokens_info, "simbolos": simbolos_json, "errores": errores,
    "tac": tac_filas, "tac_optimizado": tac_opt_filas,
    "metricas": metricas, "traza_optimizacion": traza,
}, ensure_ascii=False))
```

Qué hace: es el único archivo que une el frontend con el backend. Hace el pipeline completo *lexer* → *parser* → *generador TAC* → *optimizador* y empaqueta todo (tokens, símbolos, errores, TAC

original, TAC optimizado, métricas y traza) como un único objeto JSON que escribe a `stdout`. La UI lo invoca, lee el JSON y pinta los paneles.

7. Parte visual: pestaña de Código Intermedio en `index.php`

7.1. Lectura del JSON

```
const tac      = data.tac          || [];
const tacOpt   = data.tac_optimizado || [];
const metr     = data.metricas      || {};
const traza    = data.traza_optimizacion || [];

renderIntermediate(tac, tacOpt, metr, traza);
```

Qué hace: extrae los cuatro campos del JSON producido por `bridge.py` y se los pasa a la función de render.

7.2. Render: tablas, pretty, métricas y traza

```
function renderIntermediate(tac, tacOpt, metr, traza){
  // Tablas (op | arg1 | arg2 | dest)
  document.getElementById('ir-body-orig').innerHTML = tac.map(_irRowHtml).join('');
  document.getElementById('ir-body-opt').innerHTML = tacOpt.map(_irRowHtml).join('');

  // Vista pretty-printer (texto plano alineado)
  document.getElementById('ir-pre-orig').textContent = _irPretty(tac);
  document.getElementById('ir-pre-opt').textContent = _irPretty(tacOpt);
  document.getElementById('ir-cmp-orig-n').textContent = `${tac.length} cuádruplos`;
  document.getElementById('ir-cmp-opt-n').textContent = `${tacOpt.length} cuádruplos`;

  // Traza del optimizador
  document.getElementById('ir-traza-body').innerHTML = (traza||[]).map(t => {
    const dColor = t.delta<0 ? 'var(--green)' : (t.delta>0 ? 'var(--red)' : 'var(--t3)');
    const dStr   = t.delta>0 ? `+${t.delta}` : String(t.delta);
    return `<tr>
      <td>${t.iter}</td><td>${escH(t.pasada)}</td>
      <td>${t.antes}</td><td>${t.despues}</td>
      <td style="color:${dColor}">${dStr}</td></tr>`;
  }).join('');

  // Metricas
  const co = metr.cuad_orig ?? tac.length;
  const cp = metr.cuad_opt  ?? tacOpt.length;
  document.getElementById('ir-stats').innerHTML =
    `Cuádruplos: <b>${co}</b> -> <b>${cp}</b> &middot; ` +
    `Reduccion: <b>${metr.reduccion_pct ?? 0}%</b> &middot; ` +
    `Temporales: <b>${metr.temps_orig ?? 0}</b> -> <b>${metr.temps_opt ?? 0}</b>`;
}
```

Qué hace cada bloque:

- **Tablas:** rellena las tablas de TAC original y optimizado con una fila por cuádruplo (op, arg1, arg2, dest).
- **Pretty:** escribe la versión legible (estilo `c = $t1 o print x`) producida en Python por `formatear_tac`.

- **Traza:** una fila por pasada que hizo cambios. El delta se pinta en verde si redujo y en rojo si creció (no sucede en las pasadas implementadas pero deja la columna preparada).
- **Métricas:** tarjeta con cuádruplos antes/después, porcentaje de reducción y número de temporales.

7.3. Sub-vistas

```
function switchIRSub(sub){
  document.querySelectorAll('.ir-view').forEach(v=>v.style.display='none');
  const map = {orig:'ir-view-orig', opt:'ir-view-opt',
               cmp:'ir-view-cmp',   info:'ir-view-info'};
  const target = document.getElementById(map[sub] || map.orig);
  if(target) target.style.display = (sub==='cmp') ? 'flex' : 'block';
}
```

Qué hace: permite alternar entre cuatro vistas dentro de la pestaña: Original, Optimizado, Comparación (lado a lado) e Info. Oculta todas las vistas y muestra sólo la elegida.

8. Conclusión

La fase 2 deja el compilador con tres piezas bien diferenciadas:

- Un **frontend** (lexer + parser + `generador_intermedio`) que produce un AST y de ahí el TAC.
- Un **contrato** (`intermedio.py`, la dataclass `Quad`) que ambos lados conocen y nada más.
- Un **backend** (`optimizador.py`) con cinco pasadas clásicas que iteran hasta punto fijo.

La interfaz visual hace explícito el efecto del optimizador (tabla original vs. optimizado, métricas y traza), lo que vuelve al compilador útil también como herramienta de aprendizaje: no sólo *qué* se redujo, sino también *por qué* pasada y en qué iteración.